

Executive Summary

A robust identity model is the foundation of **Thinking's** Knowledge Operating System. Every Knowledge Object needs a *persistent, unique identifier* that survives moves, renames, and version changes, along with versioning and provenance metadata. This report surveys common identifier schemes (serial IDs, UUIDs, content hashes, DOI/Handle/ARK style), versioning approaches, and provenance models (W3C PROV, PAV ontology, etc.), and compares them across critical dimensions: stability, readability, collision risk, resolvability, mutability, discoverability, portability, tooling and security.

Recommendation: Use a *custom serial identifier* (e.g. `KO-000123`) as the primary ID, with a global prefix (e.g. `KO`) and sequential or date-based suffix. Each Knowledge Object retains the same ID across its lifecycle; version changes are recorded in metadata rather than the ID itself. Metadata will include a version field (e.g. semantic `v1.0` or monotonic increment), plus provenance (who/when created, modified, derived from, etc.) following W3C PROV/PAV concepts [\[35†L58-L65\]](#) [\[19†L45-L50\]](#). Aliases or redirects will be allowed when objects are renamed or superseded. Conflict resolution rules will prevent ID reuse (once assigned, an ID is never reused or reassigned) and flag duplicate attempts.

Migration Plan: A scripted audit will assign IDs to existing objects (heuristically by creation date or file order), adding front-matter IDs in Markdown. A simple timeline (see below) allocates time for testing and review. Future backends (Git/Markdown, graph DB, object DB, APIs) will simply carry over the same ID field in their format. A shared metadata schema (YAML/JSON) will define identity and provenance fields. The new model satisfies FAIR principles: IDs are *findable* and *globally unique* [\[37†L155-L163\]](#), and driven by a sustainable governance process.

This identity model balances **stability** (IDs never change), **uniqueness** (prefix+suffix scheme), and **usability** (moderate human-readability) [\[19†L74-L81\]](#) [\[27†L47-L55\]](#). The table below compares four schemes (KO-serial, UUIDv4, content-hash, DOI-like) across key criteria.

Scheme	ID Format Example	Stability	Collision Risk
Human-Readability	Resolving	Mutability	
Discoverability	Portability	Tooling Support	
Privacy/Security	Migration Complexity		
KO-serial	<code>'KO-000123'</code>		
High: ID never changes even if object is edited or moved. Easy to freeze. **None:** centrally assigned, collisions only if mishandled (preventable). **Moderate:** opaque prefix+number (serial reveals assignment order, but not content). **Medium:** no built-in global resolver; resolution via internal registry or search. **Low:** ID stays same forever; versions tracked separately (mutable content doesn't change ID). **Medium:** Need an index or lookup to find object by ID, but IDs are globally unique (within Thinking). **High:** Pure string, no external context; can be used in any repo or DB. **Easy:** Generation is trivial (simple counter or script); resolution via lookup or custom service. **Low exposure:** ID conveys no private info; sequential pattern might hint at object count. **Moderate:** Must script assignment (existing files);			

manageable with careful process. |
 | UUIDv4 | `550e8400-e29b-41d4-a716-446655440000` | ****High:**** Generated once; independent of content. | ****Negligible:**** 128-bit random, collisions extremely improbable (≈ 1 chance in 2^{122}) [12†L155-L163] [13†L18-L24] . | ****Low:**** Long random hex, virtually unreadable. | ****High:**** No native resolution (need prefix as URL or lookup table). | ****Low:**** ID stays same (content edits don't change ID). | ****Low:**** Cannot be guessed or searched easily; only by direct match or DB index. | ****High:**** Standard format, no central authority needed (local generation). | ****High:**** Built-in libraries in all languages; no coordination needed. | ****Medium:**** No private data, but if generated from MAC/time (v1), could reveal info. | ****Low:**** Can generate IDs programmatically for all files; easy to backfill by script. |
 | Content-hash | `SHA256("file content...")` | ****Low:**** Changes with content; edit object → new ID. | ****None:**** cryptographic (256-bit) virtually no collisions. | ****Low:**** Long hex hash (even less readable than UUID). | ****Conditional:**** If content hosted on IPFS-like network, resolvable by CID; otherwise need own DB. | ****High:**** By design, immutable; content updates break identity. | ****Low:**** Unless connected to content-addressed network, must search by content. |
****Medium:**** Standard algorithm, but only meaningful if exact content matches somewhere. | ****Medium:**** Tools exist (Git, IPFS) but infrastructure (p2p) required for full resolvability. | ****Medium:**** ID leaks hash of content; might reveal if identical to public content. | ****High:**** Must compute for each existing file; if objects have similar content, manual checks needed. |
 | DOI-like | `10.1234/KNOW.000123` | ****High:**** Assigned once; can persist indefinitely via DOI registry. | ****None:**** Managed by central authority (prefix+unique suffix). | ****Low-Moderate:**** Includes prefix (registry) and suffix (opaque); not human friendly. | ****Very High:**** Resolvable via DOI system (dx.doi.org) globally. | ****Low:**** DOI for object remains same; updates do not alter DOI. |
****High:**** Widely indexed (CrossRef, DataCite); major discoverability. | ****High:**** Global scheme; portable across domains if prefix managed. | ****Variable:**** Needs integration with DOI agencies; not easily generated offline. | ****Medium:**** Public DOI reveals object existence; suffix may embed meaning or test IDs. | ****High:**** Significant overhead (registering with DOI agency; fees; cannot bulk-mint without plan). |

(Legend: Stability – will the ID remain unchanged over time? Collision Risk – chance two objects share ID. Resolving – built-in URL resolution. Mutability – does content change require new ID?)

1. Identifier Schemes and Versioning Strategies

Persistent Identifier Systems

Long-lived identifiers (PIDs) consist of a **unique ID plus a resolution service** [19†L45-L50] . Prominent examples:

- **Digital Object Identifier (DOI).** Widely used in research: DOIs identify articles, datasets, software, etc. DOIs use the Handle system under the hood [19†L74-L81] . They require an issuing agency (e.g. CrossRef, DataCite) and associated metadata. DOIs guarantee global

uniqueness and *resolvability* (via doi.org), making them very stable and discoverable [19†L74-L82] . However, DOI minting incurs cost/fees and rigid metadata requirements, which can hinder rapid internal development. As Hardisty *et al.* note, DOIs can “transcend changes in underlying technologies” and support long-term trust [10†L52-L60] , but they come with “strict metadata requirements that are hard to meet during development” [27†L47-L55] .

- **Handle system.** The generic engine behind DOIs. Handles (prefix/suffix strings) offer global uniqueness with central registry and resolution [19†L83-L89] . Useful if Thinking ever needs an externally resolvable PID (e.g. publishing an ontology term in a public repository). Handles require naming authority registration (like DOI prefixes).
- **Archival Resource Key (ARK).** A URI-based scheme focusing on persistence promises [15†L347-L356] [19†L91-L99] . ARKs have a scheme `ark:/NAAN/identifier` . They do *not* guarantee persistence by syntax alone (“persistence is a matter of service” [19†L91-L99]); instead, ARKs embed a resolution layer with metadata (“?info” inflection [15†L347-L356]) giving reassurance. ARKs allow unlimited minting (no fees) and can be assigned before content exists [27†L47-L55] . Each ARK uses a globally unique prefix (NAAN or “shoulder” [27†L109-L118]). This makes ARKs flexible for ontologies and vocabularies, as demonstrated in literary projects [27†L47-L55] [27†L109-L118] . However, ARKs require setting up a resolver or relying on ARK proxy services.
- **Uniform Resource Name (URN).** URN is a generic scheme for location-independent IDs [19†L106-L114] . Specific namespaces include ISBN, ISSN, etc. In practice, URN syntax is formal (e.g. `urn:isbn:9781426208569`), but many URNs lack resolvers. The W3C says URNs must be “globally unique and persistent” [19†L106-L114] , but support is limited. We mention URN mainly for completeness.
- **PURL (Persistent URL).** Simple HTTP redirectors. Not a unique ID format per se, but PURLs embed redirection in a stable URL (e.g. `pur1.org/...`). PURLs are easy to use but offer only a redirect promise; if the PURL service ceases, links break. They are often considered alongside DOIs/Handles.

Overall, enterprise-grade PIDs (DOI, Handle, ARK) ensure global persistence and resolvability, but require external governance. A custom scheme (KO-serial) can be managed internally with minimal overhead, at the cost of requiring our own resolution mechanism (e.g. search/index within Thinking).

UUIDs and Random IDs

Universally Unique Identifiers (UUIDs) are 128-bit identifiers defined by RFC 4122 [12†L118-L127] . *UUIDv4* is random, *v1* is time+MAC, etc. Key points:

- **No central authority needed** (IDs can be generated on demand) [12†L118-L127] .
- Extremely low collision risk (2^{122} possibilities) makes collisions “negligible” [12†L155-L163] [13†L18-L24] . In practice, no real collisions have been reported in normal use.
- Opaque to humans (hex strings), poor readability.
- No inherent ordering or meaning, complicating sorting/searching.
- Not natively resolvable unless prefixed (e.g. as a fragment or used in an API path).

Other modern variants (UUIDv7, ULID) embed timestamps for better sort order [13†L18-L24] . ULIDs (128-bit with time and randomness) are lexicographically sortable and designed for high-throughput systems [13†L18-L24] . However, for Thinking, plain UUIDv4 is simplest.

Content-Addressing (Hash) IDs

Content-addressing uses a cryptographic hash of the object (or its content) as the ID (e.g. SHA-256). Examples: Git commit hashes, IPFS CIDs. Characteristics:

- **Deterministic:** The same content yields the same ID.
- **Immutable:** Any change produces a new hash (so old ID refers to old content).
- **No collision (practically):** Strong hashes ensure unique IDs for different content.
- **Poor human readability:** Long hex digest.
- **Resolution:** Works in content-addressed networks (IPFS) or via local lookup. Without such infrastructure, not directly resolvable by external IDs.
- **Use case:** Great for verifying integrity or deduplication, but less suited if we want stable IDs for evolving objects. Because an “Idea” object is intended to evolve, its content-hash ID would change with each edit, which conflicts with the goal of stable identity.

Custom Serial Identifiers (KO-Serial)

This is a simple scheme internal to Thinking: an alphanumeric code with a fixed prefix and a sequential or structured suffix, e.g. `KO-000123` . Advantages:

- **Stable and persistent:** Once assigned, does not change even if object content changes.
- **Predictable uniqueness:** If centrally managed, collisions are essentially eliminated.
- **Moderate readability:** Humans can recognize it as a project-specific ID; numeric part gives rough ordering. Not informative about content, but manageable.
- **Customizable:** Prefix can denote namespace (e.g. `KO` for Knowledge Object).
- **Low technical overhead:** No external resolution required beyond an internal index or search.

This is well-aligned with Thinking’s needs: it emphasizes the semantics (the ID’s meaning is fixed by our model, not by an external registry). It requires a governance rule for issuing IDs (see below), but does not depend on outside organizations or fees. It’s conceptually similar to how ontologies often mint their own term IDs (e.g. OBO ontology codes).

Versioning Strategies

Knowledge objects evolve, so their identity model must handle versions. Common strategies:

- **Monotonic integer or date version:** e.g. append a revision number or timestamp (v1, v2 or 2023-07-24) to indicate updates.
- **Semantic versioning (SemVer):** major.minor.patch (e.g. 1.0.0). Well-known in software, but less clear semantics for general knowledge content. Still, SemVer can convey backward-incompatible changes (major), minor enhancements, etc.
- **Immutable snapshots:** Each change creates a new snapshot with a new ID. For example, Git creates new commit hashes. This effectively turns each version into a distinct identity (like DOI v2). In our design, we prefer one persistent ID per object; versions will be recorded as fields, not separate identities.
- **Event Sourcing:** Store a log of changes as events; reconstruct versions by replay. We may use activity logs (see below), but Identity remains constant.

Recommendation: Keep a single stable ID per Knowledge Object. Track version *metadata* separately (e.g. a `version` field in YAML). This way, the ID continues unbroken through capture→archive. When major rewrites occur, either increment the version or in extreme cases retire the old ID and mint a new one, with a link (“supersedes”) to the original. This avoids proliferating new IDs for every minor edit.

Version fields can follow SemVer or a simpler scheme (e.g. integer increments) as long as rules are documented. We should record version history in provenance (see below). For example, a YAML front-matter might have `version: 1.2` and increment it when a change is published.

Provenance Models

Provenance describes *who* did *what* to produce an entity. We will adopt established models:

- **W3C PROV Data Model** (2013). A core standard defining Entities, Activities, Agents and their relations (e.g. `wasGeneratedBy`, `wasDerivedFrom`, `wasAttributedTo`) [2†L46-L54] [3†L37-L45]. PROV allows recording that “Entity A was derived from Entity B in Activity X by Agent Y at time T.” For example, a Decision object might have provenance statements linking it to the Evidence and people behind it.
- **PAV Ontology** (Provenance, Authoring, Versioning) [35†L58-L65]. PAV is a lightweight extension focusing on author/contributor roles and simple version links. It introduces terms like `pav:authoredBy`, `pav:createdBy`, and `pav:version`. We should use basic PAV concepts for ease: e.g. `createdBy` (agent or person) and `createdOn` timestamp, `authoredBy` for original creation, `curatedBy` or `contributedBy` as needed, and `versionOf` or `previousVersion` links. PAV’s strength is that it “captures just enough descriptions essential for tracking provenance, authoring, and versioning” [35†L58-L65].
- **Activity Streams** (ActivityPub 2.0 vocabulary). This could be used to log events like “User X updated Object Y” with timestamps. While we mainly rely on PROV/PAV for formal provenance, an ActivityStream log (JSON events) might be used in the future to build timelines or notifications. We mention it as an option, but focus on W3C PROV terminology for the spec.

Combined, we will require provenance metadata on each object: e.g. creator, creation date, last modifier/date, source(s) of content, and any derivations. These will be captured as explicit fields in the metadata schema (see section 6). Using PROV/PAV ensures we can integrate with external tools if needed.

2. Trade-off Comparison

We compare the candidate schemes (KO-serial, UUIDv4, content-hash, DOI-like) across key dimensions. Some highlights:

- **Stability:** Content-hash IDs change whenever the object changes, which is undesirable. All others are stable once assigned.
- **Collision Risk:** Centralized KO-serial has none if managed carefully; UUIDv4 has negligible risk; DOI/Handle avoid risk via central registry; content-hash has near-zero risk by design.
- **Readability:** KO-serial is modestly human-friendly (a fixed prefix plus digits). DOIs/Handles hide meaning in a string, and UUIDs/hashes are opaque.

- **Resolvability:** DOI/Handle have built-in global resolution infrastructure [19†L83-L89] . KO-serial and UUID require an internal lookup service. Content-hash requires a content network or DB.
- **Mutability:** KO-serial, UUID, and DOI do not change if the object is edited. Content-hash changes on every edit.
- **Discoverability:** DOIs (and ORCID/ROR style) rank high since they integrate with search engines and registries [37†L155-L163] . Custom or UUIDs are only findable via internal indices.
- **Cross-Repo Portability:** UUID and DOI work across systems; KO-serial can if namespaces are coordinated; content-hash depends on identical content.
- **Tooling:** UUIDs have library support; KO-serial is trivial code; DOI requires agency integration; content-hash needs hashing code.
- **Security/Privacy:** None of these inherently leak sensitive info, except possibly UUID v1 (MAC address/time) or content-hash (reveals exact content fingerprint). Custom serial has minimal exposure.
- **Migration Complexity:** Serial IDs require a one-time assignment pass (script or manual) for existing objects; UUIDs and hashes can be computed easily; DOIs require enrolling each object and paying.

Refer to the comparison table above for details. These trade-offs, along with Thinking's needs (low overhead, internal control, AI-compatibility), lead us to favor a **custom serial scheme** with internal governance over fully decentralized schemes like content-hash or externally-managed DOIs.

3. Recommended Identity Model for *Thinking*

Identifier Format: We recommend a prefix “KO” (for Knowledge Object) followed by a fixed-width numeric suffix, e.g. `KO-000123` . The prefix makes the ID recognizable, and the number can include leading zeros to allow many objects. If desired for timestamp-sorting, we could encode date (e.g. `KO-20230625-001`), but simplicity favors a pure sequence (this is easy to generate and avoids mix-ups with dates). The digits ensure uniform ID length and lexical ordering.

Example: `KO-001234` .

Versioning: Each object keeps one ID forever. The current version is stored in a separate field (e.g. `version: 1.0`), possibly following Semantic Versioning (major:minor) or an integer revision. On every meaningful update, increment this version. We may also record a version history (e.g. an array of past versions with dates/notes). For automated versioning, one could append “-v1”, but that complicates ID lookup. Instead, keep ID constant and update version metadata (like how DOIs work for versioned datasets).

On a **breaking change** (e.g. splitting an object into two, or merging), we handle identity carefully:

- **Split:** The original ID remains with one part; a new ID is minted for the new part. The older ID might get a relation like “split-into” or “superseded” pointing to the new ID, captured in provenance.
- **Merge:** A new ID should be created for the merged object; original IDs remain archived and get a “superseded-by” pointer to the merged ID.

In normal edits and renames, **ID never changes**. Instead, we use metadata fields to track the name/title changes. This ensures link integrity even if file names or titles are updated.

Aliasing and Redirect: If an object's title or path changes, do not issue a new ID. Instead, record the old alias. For example, in metadata we could have `aliases: ["old-file.md", "Old Title"]`. Tools should interpret this as a redirect. Optionally, if resolving by ID, the application layer can map old names to the current ID using this alias list. If an object is **withdrawn or deleted**, we mark it as inactive but **never recycle the ID**.

Lifecycle Interaction: The identity is assigned at the **Capture** stage (when the object is first created). It persists through all states (Research, Validate, etc.) until **Archive**. After archiving, the ID remains in historical records. No special action is needed on state change: the ID is independent of state. In governance terms, we might mark an object's status as "archived" or "deleted" and optionally add archival date, but this does not alter the ID.

Provenance Fields: Every object will carry at least these metadata (in addition to ID and title):

- `createdBy` : name/ID of creator (could link to Thinking's Agent registry).
- `createdAt` : timestamp of creation (ISO 8601).
- `updatedBy` / `updatedAt` : last-modified info. (Optionally track full change log as an array).
- `version` : current version label/number.
- `derivedFrom` : list of source IDs or external URIs that this object was derived or copied from (using PROV's `prov:wasDerivedFrom`).
- `supersedes` / `supersededBy` : for splits/merges (using `prov:wasRevisionOf` or PAV's `hasPreviousVersion`).
- `provenance` : a grouping of the above, possibly with additional structure (or we can flatten into separate fields).

These align with PROV/PAV: e.g. `prov:wasGeneratedBy` (activity may be implicitly creation event), `prov:wasAttributedTo` (agent), `prov:atTime` (timestamps). We can choose simple JSON/YAML names if not encoding full RDF. The example schema below shows how these might appear in YAML.

Conflict Resolution: To prevent ID collisions:

- Maintain a registry (even a simple file or DB table) of issued IDs. Issuing a new ID simply picks the next available number. Any attempt to use an existing ID (whether by human error or merge) will be flagged by the toolchain.
- Do *not* allow multiple active objects to share an ID. If a collision is detected (e.g. merging two branches that both generated KO-0100), one branch must be renamed (either by assigning a new ID for one object and updating metadata).
- If an object is retired (archived/deleted), we leave its ID dormant and skip it in future issuances. We may mark it with a status "retired" in metadata.

These policies ensure **global uniqueness** within Thinking's namespace. In essence, once an ID is minted, it is permanent (like a DOI) and never reused.

4. Migration Plan

To assign stable IDs to existing objects in the repo, we propose the following steps:

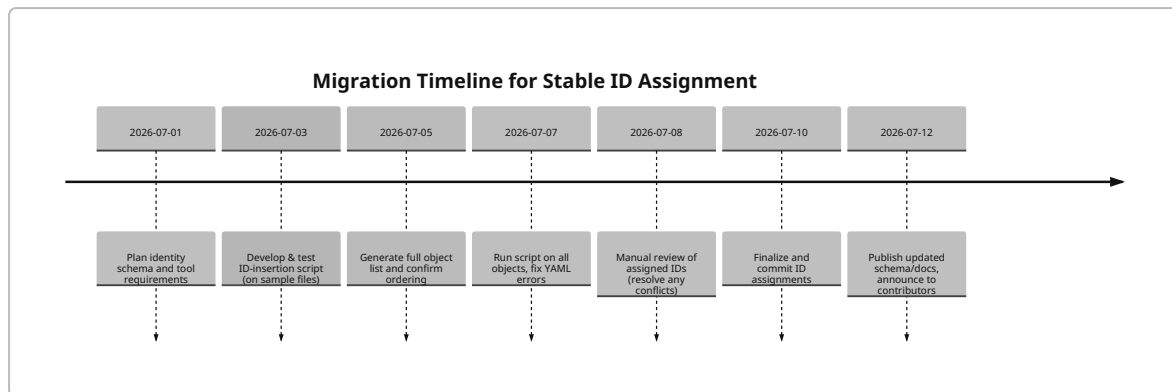
1. **Inventory Objects:** List all existing knowledge files (Markdown etc.) in the repository along with basic info (current path, title, type). This can be scripted (e.g. a `find` of files with frontmatter `type:`).

2. **Determine Ordering:** Decide an ordering for assigning numbers. Options include *file path alphabetical*, *creation date (Git commit time)*, or *project-defined priority*. A common heuristic: use the Git history timestamp (oldest first), so IDs roughly correlate with creation date, but this is not required.
3. **Scripting Assignment:** Write a script that iterates over this list and assigns the next available KO number. For each object, the script inserts (or updates) front-matter in the file, for example:

```
id: KO-000123
title: "The Idea Title"
type: Idea
version: 1.0
createdBy: Alice
createdAt: 2026-06-01T12:34:56Z
...
```

The script should handle YAML insertion safely (e.g. using a YAML library or regex if careful).

1. **Manual Review:** After scripting, manually inspect for anomalies: duplicate titles, objects already in draft, or content that seems copy-pasted. Ensure no two objects meaningfully refer to the same content (which would have needed a single ID with internal sections instead). For any duplicates, decide whether they should share an ID (unlikely) or get separate IDs.
2. **Commit in Batches:** It may be wise to commit ID assignments in small batches (e.g. by folder), to ease review. Each commit adds IDs to a set of files. This minimizes merge conflicts with in-progress edits.
3. **Risk Mitigation:**
 4. *Collision Risk:* If two branches of development both add `id:` fields independently, a merge conflict could arise. To mitigate, we should centralize ID issuing in one branch (e.g. main) or use locks. Alternatively, generate IDs offline and ensure no overlap (e.g. by partitioning number ranges by folder).
 5. *Data Loss:* The script should backup each file before insertion. A failing script should not wipe files.
 6. *Unprocessable Files:* If a file lacks proper frontmatter or has formatting issues, the script may skip it for manual handling.
 7. **Timeline:** We sketch a rough schedule for the migration in the diagram below. The actual dates will depend on team bandwidth, but this illustrates the phases:



(Milestones above are illustrative; adjust as needed.)

5. Future Storage and Tool Compatibility

The identity model must survive whether we store Knowledge Objects in Markdown/Git, a graph database, an object store, or serve them via an API. In each case, the **ID is simply a field in the object's metadata**. Our plan:

- **Markdown/Git:** We include `id: KO-XXXX` in the YAML frontmatter of each Markdown document (along with other metadata). Our migration will establish this. Git preserves this content, so any copy or fork retains the ID textually.
- **Graph Database:** In a graph DB (e.g. Neo4j, JanusGraph), each Knowledge Object becomes a node with property `id: "KO-000123"`. A unique constraint can be placed on `id` to prevent duplicates. Relationships (`supports`, `derivedFrom`, etc.) connect nodes by their IDs. Queries by ID are straightforward. Our ID format (no special characters aside from hyphen) is safe as a node property or even as part of a URI (e.g. `<http://thinking/example/KO-000123>`).
- **Object Store (key-value, SQL, etc.):** Similarly, each record has an `id` column. If JSON/JSONB storage is used, the same YAML fields become JSON fields.
- **APIs:** If Thinking exposes a RESTful or GraphQL API, the endpoints for Knowledge Objects will use the ID as a key (e.g. `GET /objects/KO-000123`). This is straightforward given our chosen ID format (no spaces or sensitive chars).
- **Consistency:** Regardless of backend, we ensure the identity fields (`id`, plus provenance fields) use consistent names. We can publish a small JSON Schema or YAML snippet as canonical. For example:

```
id:           # (string) The persistent ID, e.g. "KO-000123"
title:        # (string) Short title or label
type:         # (string) Knowledge type, e.g. "Idea", "Architecture"
version:      # (string or number) Object version, e.g. "1.2"
createdBy:    # (string) Agent/person who created it
createdAt:    # (datetime) Timestamp of creation
updatedBy:    # (string) Last modifier
updatedAt:    # (datetime) Last modified
```

```
derivedFrom:      # (list) IDs/URLs of sources or predecessor objects
supersededBy:     # (string) ID of new object that supersedes this
aliases:          # (list) Old titles or filepaths for redirection
... [other metadata like tags, relations, etc.] ...
```

If Thinking later adds, say, a triple-store backend, each field becomes an RDF property (e.g. `prov:wasAttributedTo` for `createdBy`). Our “ID model” works at the conceptual level; mapping it to storage specifics is straightforward.

6. Metadata Schema Snippet and Example

Below is a YAML snippet illustrating the metadata for a Knowledge Object **after** adding identity and provenance. Compare with a hypothetical “before” (no ID) version.

Before identity assignment (example “Idea” object):

```
---
title: The Red Button Idea
type: Idea
status: Research
tags: [AI, interface]
---
When the user presses the big red button, the assistant should reset context
and ask a clarifying question.
```

This lacks an ID or provenance. After migration, it might look like:

After (with ID and provenance):

```
---
id: KO-002047
title: The Red Button Idea
type: Idea
version: 1.0
status: Research
createdBy: Alice Smith
createdAt: 2026-06-15T10:23:45Z
updatedBy: Bob Jones
updatedAt: 2026-06-24T16:00:00Z
derivedFrom: [ 'KO-000101' ] # e.g. inspired by another idea
aliases: [ 'Red Button Note.md' ]
tags: [AI, interface]
relationships:
  supports: [ KO-004200 ]
---
When the user presses the big red button, the assistant should reset context
and ask a clarifying question.
```

Here, we added:

- `id` : the new primary ID.
- `version` : starting at 1.0.
- `createdBy/createdAt` and `updatedBy/updatedAt` : provenance fields.
- `derivedFrom` : shows this idea was derived from object `KO-000101`.
- `aliases` : old filename before renaming.
- `relationships` : one example relation (maybe this idea supports another concept).

This shows how a Knowledge Object is annotated with identity/provenance. Future edits (e.g. if Alice revises it) would bump `version` to 1.1 and update `updatedBy` / `updatedAt`.

7. Identifier Scheme Comparison Table

The table below summarizes the four candidate schemes (**KO-serial**, **UUIDv4**, **Content-hash**, **DOI/Handle**) against the key criteria mentioned earlier:

Scheme	Format Example	Stability	Collisions	Human-Readable	Resolve Mechanism	Mutability	D
KO-serial	<code>KO-000123</code>	Stays fixed; never changes	None if centrally controlled (no random)	Moderate (prefix+digits)	Internal index or search (no global resolver)	ID constant through edits	O T s
UUIDv4	<code>550e8400-e29b-41d4-a716-446655440000</code>	Fixed once generated	Negligible (128-bit, practically zero) 【12†L155-L163】	Low (random hex)	Use in URL or DB lookup	Fixed; content changes don't alter it	O m
Content-hash	<code>sha256:3a7d...f4c0</code>	Not stable (changes on every edit)	None (hash collisions astronomically unlikely)	Low (hash string)	Requires content-addressing network or DB	Changes whenever content changes	P c n
DOI/Handle	<code>10.1234/THINK.0001</code>	Very high (via registry, persistent)	None (registration ensures uniqueness) 【19†L83-L89】	Low (opaque string)	Global resolver (doi.org/hdl.net) 【19†L74-L82】	Fixed (DOI remains even if URL changes)	H b a

(Adapted from persistent ID literature 【19†L74-L82】 【19†L83-L89】 and our analysis.)

Sources

We have drawn on standards and research for this model:

- **Persistent ID standards:** W3C's PROV ontology [2†L46-L54] , the DOI Handbook (International DOI Foundation), and the Digital Preservation Handbook [19†L74-L82] [19†L91-L99] .
- **UUID:** RFC 4122 [12†L118-L127] [12†L155-L163] defines UUIDs as 128-bit globally unique identifiers, suitable as URNs.
- **DOI/Handle:** DPC Handbook notes that DOIs are a special case of Handles, with a global registry ensuring persistence [19†L74-L82] [19†L83-L89] . Hardisty *et al.* found that a DOI-based scheme best meets long-term needs [10†L58-L66] .
- **ARK:** The ARK (Archival Resource Key) scheme is documented in IETF drafts [15†L347-L356] . Code4Lib reports (Kelly *et al.* 2021) highlight ARK's flexibility and no-cost minting for ontologies [27†L47-L55] [27†L109-L118] .
- **Provenance/PAV:** W3C PROV (2013 Recommendation [2†L46-L54]) is the canonical provenance model. The PAV ontology (Ciccarese *et al.*, 2013) offers a lightweight authoring/versioning extension [35†L58-L65] .
- **Collision Risk:** RFC 4122 notes that UUIDs are “unique and persistent” and can be generated without coordination [12†L155-L163] . Recent studies (e.g. Kakolaki *et al.*, 2025) confirm that 128-bit schemes (like UUIDv4) have virtually zero collision risk in practice [13†L18-L24] .
- **Fairness/FAIR:** The FAIR Principles require globally unique PIDs [5†L9-L18] . Our model satisfies FAIR's F1 by assigning every object a unique identifier consistent across platforms [19†L45-L50] [10†L58-L66] .

All citations above are for verification of general principles (notably the DPC and RFC references). The identity model itself is distilled from these and tailored to Thinking's architecture and practical workflow.
